You can use any of the standard querying methods inside the scope block. The following ones are discussed below:

- `where`
- `includes`
- `readonly`
- `select`

4.2.3.1 `where`

The `where` method lets you specify the conditions that the associated object must meet.

```
class Supplier < ActiveRecord::Base
  has_one :account, -> { where "confirmed = 1" }
end
```

4.2.3.2 `includes`

You can use the `includes` method to specify second-order associations that should be eager-loaded when this association is used. For example, consider these models:

```
class Supplier < ActiveRecord::Base
  has_one :account
end

class Account < ActiveRecord::Base
  belongs_to :supplier
  belongs_to :representative
end

class Representative < ActiveRecord::Base
  has_many :accounts
end
```

If you frequently retrieve representatives directly from suppliers (`@supplier.account.representative`), then you can make your code somewhat more efficient by including representatives in the association from suppliers to accounts:

```
class Supplier < ActiveRecord::Base
  has_one :account, -> { includes :representative }
end

class Account < ActiveRecord::Base
  belongs_to :supplier
  belongs_to :representative
end

class Representative < ActiveRecord::Base
  has_many :accounts
end
```

4.2.3.3 `readonly`

If you use the `readonly` method, then the associated object will be read-only when retrieved via the association.

4.2.3.4 `select`

The `select` method lets you override the SQL `SELECT` clause that is used to retrieve data about the associated object. By default, Rails retrieves all columns.

4.2.4 Do Any Associated Objects Exist?

You can see if any associated objects exist by using the `association.nil?` method:

```
if @supplier.account.nil?
  @msg = "No account found for this supplier"
end
```